

GC.OS Proposal :Embedded AI for Predictive Sensor Systems in Agriculture 4.0

Size: Big and Applied Project

Personal Details

- **Name:** Ronak Raj
- **Email:** codeitronak226277@gmail.com
- **GitHub:** <https://github.com/RONAK-AI647>
- **LinkedIn:** <https://www.linkedin.com/in/ronak-r-a221a7386/>
- **Time Zone:** India (UTC+5:30)
- **University:** BIT Sindri
- **Degree:** Bachelors of Technology
- **Major:** Artificial Intelligence and Material Sciences
- **Hub:** German Center for Open Source AI
- **Agri4-sktime-repo:**

<https://github.com/RONAK-AI647/agri4-sktime-benchmark>

Brief Bio:

I am an Artificial Intelligence Researcher , Software developer and an upcoming Kubecon Speaker where I will present research on Embodied Intelligence and Edge Computing architectures. Showcasing advanced methodologies for integrating Vision-Language Models (VLMs) with decentralized edge orchestration . I am passionate about AI ,LLMs , RAGs and natural sciences. I love learning applied AI on natural sciences and do open source contributions. I have contributed to open source for more than 3 years and a top 10 contributor in google-deepmind/torax (3Prs),CNCF/kubeEdge(6Prs), Langchain-ai/langchain-google (10Prs) , Learning-Equality(10Prs) etc where I contributed in plasma research , embodied intelligence benchmarks and dataset, genai and vertex-ai and gemini API integrations by adding multimodal features and fixing streaming issues.

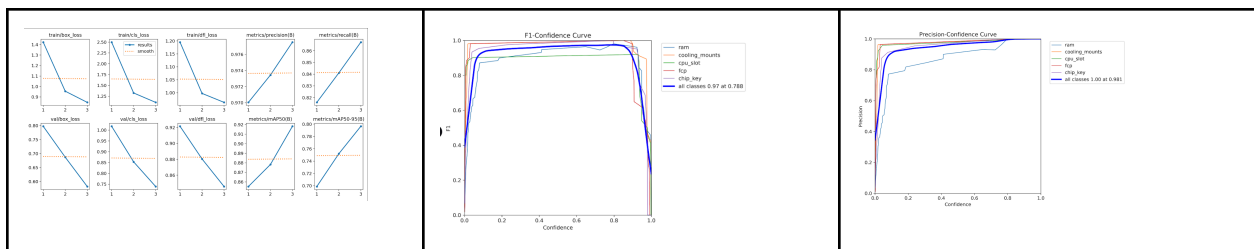
.....

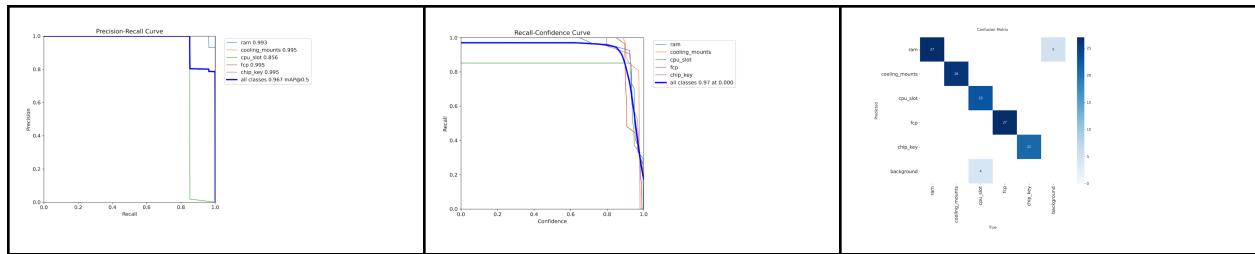
Because I deliver complex milestones on schedule , add innovation to my work , while maintaining proactive , transparent communication demonstrated by my contributions:

Contributed in KubeEdge/Ianvs , a Distributed Synergy AI Benchmarking Suite for which I created a novel industrial grade dataset based on EAI where a robotic arm assembles components on a motherboard and did its benchmarking with ianvs fetching an accuracy of 64%. ([PR](#))



- ☐ I produced the benchmarking results in beautiful tabular forms.
- ☐ Added matplotlib Curves to analyse the performance analysis myself
- ☐ In addition to this , the robot, motherboard , components , the lab setup , everything was designed by me which **impressed** the mentors , and they invited me to add this in the core code of Ianvs .





Skills used : Pytorch, transformers ,benchmarking, py-bullet, blender, matplotlib, numpy etc

Language: Python

Algorithms : YOLOv8, force/torque sensors

"The same skills — building a benchmarking suite, designing datasets, producing reproducible evaluation results — are exactly what this agriculture project requires."

- **Top 10 Contributor in Google-deepmind/torax:** Got 3 PRs merged, where I contributed to torax(14 commits : 219++ 516--)
Skills used: JAX , plasma physics
- **Top contributor to Langchain-ai/lanchain-google:** 6 PRs merged in genai. Vertai Packages
Skills used: gemini API

Maintainers and collaborations:

hamelphi commented on Feb 4 Collaborator ... Your work was great and very useful. Thanks! 1	Hamel Phillipe-Engineer,google deepmind
Zimu Zheng • 1st Head Research Engineer of Huawei Cloud 1mo • Congrats! Exciting to see the impressive work done by Ronak from KubeEdge SIG AI. See the released ... more	Zimu Zheng- Head Research Engineer of Huawei Cloud

“What did I learn?”

- Add quality code and change the minimal and most effective in a big codebase, our one change breaks 10 unit tests
- Use AI to boost our contribution , not bloat the repo with unnecessary code

Why this Project?

“Embedded AI for Predictive Sensor Systems in Agriculture 4.0”

The description of this project resonated immediately with me because it is asking for exactly the kind of work I have already been doing independently as preparation — not as a checkbox exercise, but because I found it genuinely interesting.

The core challenge — detecting foreign object events in a continuous multi-sensor stream, distinguishing them from thousands of hours of normal operating conditions — is a hard real-world problem. It is not a toy benchmark. The sensors are fast (millisecond granularity), the events are rare, and the cost of a false negative (missed detection, machine damage) is completely different from the cost of a false positive (unnecessary stop). That asymmetry makes it technically interesting and practically meaningful.

This project is about 2 things:

- use sktime’s mature parts (classifiers, transformations, pipelines) for the agriculture pipeline
- AND build the missing detection-specific infrastructure (metrics, benchmarking, multivariate detectors) as extended capabilities of sktime .

My Preparation and Contribution:

I have been trying to understand sktime framework and contributing accordingly since weeks: I am listing all the activity so far below, in further proposal I will explain what and why for my actions:

PRs Raised:

[ENH] add MultiChannelDetectionDelay metric for multi-channel event detection	https://github.com/sktime/sktime/pull/993 9
[ENH] Add performance warning in ClaSPSegmentation for large time series input	https://github.com/sktime/sktime/pull/10101

[ENH] Add multivariate support to TimeSeriesForestClassifier	https://github.com/sktime/sktime/pull/10102
---	---

Issues Raised:

[ENH] TimeSeriesForestClassifier Multivariate Support #9893	https://github.com/sktime/sktime/issues/9893
[ENH] Add windowed time series evaluation utilities for overlapping window	https://github.com/sktime/sktime/issues/9894
[BUG] ClaSPSegmentation: no performance warning for large time series input	http://github.com/sktime/sktime/issues/10099
[BUG] STRAY detector does not implement predict() — breaks standard detector API	https://github.com/sktime/sktime/issues/10100

PRs Reviewed:

[ENH] add DetectionDelayMean metric for average detection delay	https://github.com/sktime/sktime/pull/9895
--	---

Rather than just reading about the project, I spent the past few days building a working **surrogate pipeline** and making real contributions to **sktime**. Here is what I actually did.

The dataset I am gonna be provided is a unique dataset , it has multiple state of the art sensors included (e.g. vibration, acoustics and mechanics) and measured with millisecond granularity. I wanted to check if sktime can evaluate such a dataset.

So, I used the **Environmental Sensor Telemetry dataset** which I downloaded from **kaggle** [see repo under data/] — 405,000 rows of continuous IoT readings from 3 devices, with CO, humidity, temperature, smoke, LPG, and motion sensors, sampled every 5-10 seconds. I know this is not the same dataset, but it has certain features which I wanted to test and check their integration with sktime. I chose this specifically because it mirrors the structural properties of the agricultural dataset: multiple sensors, continuous high-frequency readings, and event labels (motion detection) embedded in a sea of normal readings.

Dataset kaggle link:

<https://www.kaggle.com/code/rjconstable/environmental-sensor-telemetry-dataset/notebook>

I built a full 4-notebook pipeline: [see repo link]

Notebook 01 — Data Loading and Format Conversion. I loaded the raw CSV and converted it to sktime's panel format (MultiIndex with device and timestamp). I ran `check_raise` against the `pd-multiindex` mtype and confirmed the data passed sktime's validation. This alone surfaced a real issue: the timestamp column was in Unix float scientific notation and required non-trivial conversion, and the CSV header row was malformed, requiring manual column specification. I documented both in a `roadblocks.md` file.

Notebook 02 — EDA and Visualization. I visualized temperature signals per device over the full week-long recording, plotted motion event timing as scatter markers across all three devices, and produced a summary of event counts. This gave me concrete intuition about what "normal" looks like versus event-adjacent readings.

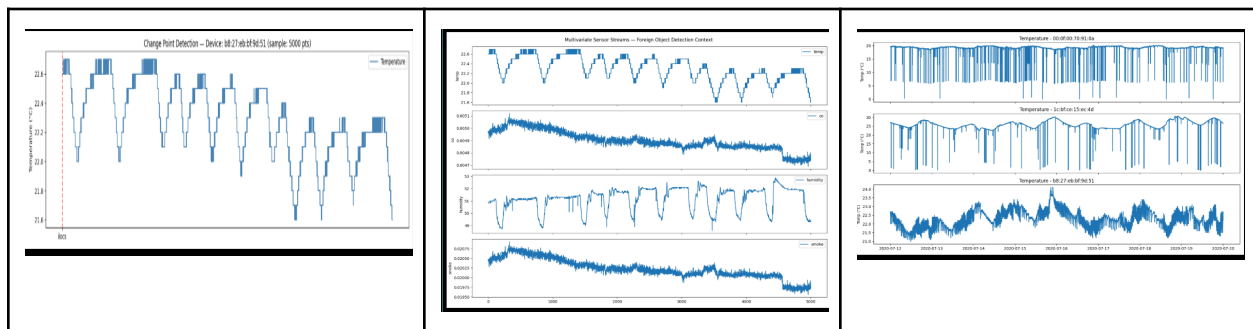
Notebook 03 — Anomaly and Change Point Detection. I ran `ClaSPSegmentation` on a 5,000-point subsample of the temperature signal (subsampling required because of a performance issue I describe below). ClaSP detected 4 structural change points. I then overlaid these against the motion event series, producing a two-panel plot showing that detected change points cluster near periods of elevated activity — which is the kind of correlation we want from a detection system.

Notebook 04 — Benchmarking. I benchmarked ClaSP against STRAY on the same segment. ClaSP: 22.88 seconds, 4 detections. STRAY: 1.04 seconds, 0 detections (requiring parameter tuning). This comparison is exactly what the project description means by "rapidly test and evaluate algorithmic approaches."

All code, plots, and the roadblocks log are in my public repository:
github.com/RONAK-AI647/agri4-sktime-benchmark

I ran two sktime detectors on real data and produced a comparison table and plots:

Algorithm	Runtime_seconds	Detections	Detection_Type
ClaSPSegmentation	22.88	4	Change Points
STRAY	1.04	0	Anomaly Flags



I did this entire demo examination to check sktime's capabilities, and check what's missing !!!

While working on the pipeline above, I encountered **real issues in sktime**. I did not go looking for issues to report — I hit them naturally while doing the work.

Performance warning in ClaSPSegmentation for large series:

Running ClaSP on the full 187,000-point series caused the process to run for 48+ minutes with no output, no warning, and no way to estimate completion time. For a real agricultural dataset with continuous sensor streams, this is a serious usability problem. I added a `UserWarning` inside `_fit()` that fires when the series length exceeds 10,000 points, with a message that suggests subsampling and references `find_dominant_window_sizes` for period length selection

Pull Request #10101 — Performance warning in ClaSPSegmentation for large series.

Link: github.com/sktime/sktime/pull/10101

Multivariate support for TimeSeriesForestClassifier.

`TimeSeriesForestClassifier` was tagged `capability:multivariate: False` while `RocketClassifier` and `ShapeletTransformClassifier` both support multivariate input. This inconsistency directly blocks multi-sensor agricultural workflows. I studied how `RocketClassifier` handles multivariate — it uses a `use_multivariate` parameter with "auto", "yes", "no" options and routes to different internal transforms via `_X_metadata`. I implemented the same pattern for `TimeSeriesForestClassifier`:

- Added `use_multivariate` parameter with `"auto"` as default, so no existing univariate code breaks
- Added `_transform_multivariate` in the base forest class — extracts mean, std, and slope features per channel independently, then concatenates, which is structurally equivalent to how ROCKET applies convolutions across channels
- Updated `_tags` with `"capability:multivariate": True`
- Added 4 tests: shape correctness, end-to-end fit/predict on multivariate data, invalid parameter validation, and a regression test confirming univariate behavior is unchanged

PR link: github.com/sktime/sktime/pull/10102

Detection delay Mean metric and Multisensor Channel metric:

When I first read the agriculture project description carefully, one thing was immediately clear to me — this is **not a single-sensor problem**. The sponsor's machinery has vibration sensors, acoustic sensors, mechanical sensors, all firing together at millisecond granularity. And the project specifically asks for advance detection time as a metric. So from the very beginning, before I wrote a single line of code, I knew that the evaluation side of this project needed something sktime did not yet have — a way to measure how early a detection fires, and a way to measure that across multiple sensor channels simultaneously.

Before I could raise an issue about the missing advance detection time metric in sktime, I found Aditya had already opened PR #9895 — **DetectionDelayMean**. His work was solid, but reviewing it carefully I caught two things. First, a logic gap — early detections within the tolerance window were being silently ignored, treated as misses. I flagged it, and Aditya fixed it. Second, something more fundamental — the metric only handled one sensor at a time, but this project has vibration, acoustic, and mechanical sensors all firing together. A single-channel metric cannot answer whether the *system* detected the event in time across all channels at once.

So I built that myself — PR #9939, **MultiChannelDetectionDelay**. It handles any number of channels and aggregates per-channel delays into one score using mean, min, max, or weighted aggregation depending on what the use case needs. This is the metric I will plug directly into my benchmarking harness for the agriculture project — every algorithm I test will be evaluated not just on TPR and FPR, but on how many milliseconds in advance it warned, across all sensors simultaneously.

PR link: <https://github.com/sktime/sktime/pull/9939>

Meanwhile, I opened issues for all the small bugs I faced:

- Issue #10099 — ClaSP large series performance warning (closed by PR #10101)

- Issue #10100 — **STRAY detector** does not implement ``predict()``, breaking the standard ``BaseDetector`` API contract. ``fit_transform()`` works but this inconsistency

prevents writing generic detection pipelines that swap algorithms interchangeably — which is one of sktime's core design goals. I plan to implement `predict()` for STRAY during the project period.

- Issue #9893 — TimeSeriesForestClassifier multivariate support (closed by PR #10102)

Conclusion: I get an idea of what sktime has and what's missing for this project?

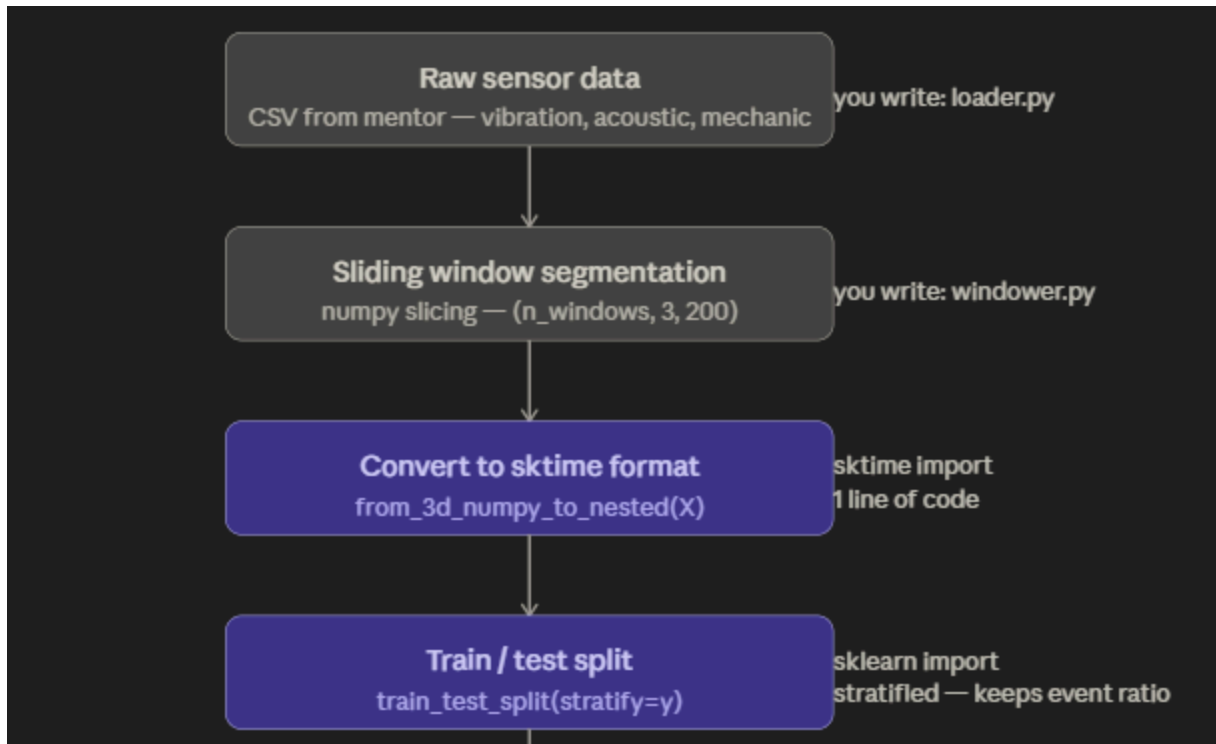
what sktime HAS (mature)	what sktime is MISSING (detection)
benchmarking for FORECASTINGForecastingBenchmark — MASE, RMSE, MAE	benchmarking for DETECTION-no DetectionBenchmark class exists
metrics for FORECASTINGMeanAbsoluteError, MASE, CRPS	metrics for DETECTIONNT PR, FPR exist — advance time does NOT
algorithms for CLASSIFICATIONRocket, TSF, HIVE-COTE2, DrCIF — mature	algorithms for DETECTION STRAY, ClaSP — experimental, incomplete AP I
evaluation for CLASSIFICATION accuracy, F1, confusion matrix — done	evaluation for DETECTION advance detection time — issue #9887 open now

Honest assessment of preparation work

I want to be direct about something. The real dataset is going to be harder than my surrogate — more sensors, faster sampling, more subtle event signatures. I am not pretending my surrogate work is equivalent. What it proves is that I know how to set up the pipeline, I know where sktime's edges are, and I know how to contribute fixes when I hit them. That is the preparation I can offer

What's my approach for the project ??

Firstly manipulate the data { we never know the dataset format}



I will extend this to a complete project pipeline:

```

4-sktime-benchmark/ |
├── data/
│   ├── raw/          ← sensor CSV from manufacturer
│   └── processed/     ← windowed numpy arrays
├── src/              | agri4/
│   ├── __init__.py
│   ├── data/
│   │   ├── __init__.py
│   │   ├── loader.py    ← raw CSV → pandas MultiIndex
│   │   ├── windower.py  ← stream → (n_windows, n_sensors, length)
│   │   └── formatter.py  ← numpy → sktime nested DataFrame
│   ├── detection/
│   │   ├── __init__.py
│   │   ├── supervised.py ← RocketClassifier, TSForest wrappers
│   │   ├── semisupervised.py ← MVCAPA, STRAY wrappers
│   │   └── baseline.py   ← threshold, IsolationForest
│   └── evaluation/
│       ├── __init__.py
│       ├── metrics.py    ← TPR, FPR, advance_detection_time()
│       └── benchmark.py  ← DetectionBenchmark class ← my sktime PR
  
```

```

— pipeline.py          ← wires everything together

notebooks/            | |— 01_data_loading.ipynb
— 02_eda.ipynb
— 03_anomaly_detection.ipynb
— 04_benchmarking.ipynb

sktime_contributions/ | |— detection_benchmark.py ← generalised, no agri-specific code

experiments/
— results/
  |— benchmark_results.csv |
tests/
— test_loader.py
— test_metrics.py
— test_benchmark.py

requirements.txt
README.md
roadblocks.md

```

This has three very important files:

src/agri4/data/loader.py — this is the dataset integration with sktime:

```

import pandas as pd
import numpy as np
from sktime.utils.data_container import from_3d_numpy_to_nested

class SensorDataLoader:
    """
    Loads raw sensor CSV and converts to sktime panel format.
    Works with surrogate IoT dataset now.
    Will work with real agricultural dataset unchanged.
    """
    def __init__(self, sensor_cols, label_col, window_size=50, stride=25):
        self.sensor_cols = sensor_cols
        self.label_col = label_col
        self.window_size = window_size
        self.stride = stride

    def load_raw(self, path):
        df = pd.read_csv(path,
            names=['ts', 'device', 'co', 'humidity',
                'light', 'lpg', 'motion', 'smoke', 'temp'],
            skiprows=1)
        df['ts'] = pd.to_datetime(df['ts'].astype(float), unit='s')
        df = df.sort_values(['device', 'ts'])

```

```

df = df.set_index(['device','ts'])
return df

def to_sktime_panel(self, df):
    """Core method — converts raw df to sktime nested DataFrame + labels"""
    windows, labels = [], []
    for device, group in df.groupby(level='device'):
        vals = group[self.sensor_cols].values # (n, n_sensors)
        lbls = group[self.label_col].astype(int).values
        for i in range(0, len(vals)-self.window_size, self.stride):
            windows.append(vals[i:i+self.window_size].T) # (n_sensors, wlen)
            labels.append(int(lbls[i:i+self.window_size].any()))

    X = np.array(windows) # (n_windows, n_sensors, window_size)
    y = np.array(labels)
    return from_3d_numpy_to_nested(X, y) # ← sktime format

```

- `src/agri4/evaluation/metrics.py` –the advanced metric detection pr after merging

```

import numpy as np

def advance_detection_time(y_pred, event_onsets, stride_ms=100):
    """
    Computes advance detection time distribution.
    This is missing from sktime
    """
    advances = []
    for onset in event_onsets:
        onset_win = onset // stride_ms
        search = y_pred[max(0, onset_win-50):onset_win]
        if search.sum() > 0:
            first = np.argmax(search > 0)
            advances.append((len(search) - first) * stride_ms)
    if not advances:
        return {"mean_ms": 0, "p90_ms": 0, "detection_rate": 0}
    return {
        "mean_ms": round(float(np.mean(advances)), 1),
        "p90_ms": round(float(np.percentile(advances, 90)), 1),
        "detection_rate": round(len(advances) / len(event_onsets), 3)
    }

```

`src/agri4/evaluation/benchmark.py` — my DetectionBenchmark prototype:

```

import pandas as pd

```

```

from sklearn.metrics import recall_score, precision_score
from sklearn.model_selection import train_test_split

class DetectionBenchmark:
    """
    Prototype of what will become sktime's DetectionBenchmark.
    Fix dataset + metrics once. Swap algorithms freely.
    """
    def __init__(self, X, y, event_onsets=None, test_size=0.2):
        self.X_train, self.X_test, self.y_train, self.y_test = \
            train_test_split(X, y, test_size=test_size,
                            stratify=y, random_state=42)
        self.event_onsets = event_onsets
        self.results = []

    def run(self, name, model, supervised=True):
        if supervised:
            model.fit(self.X_train, self.y_train)
        else:
            model.fit(self.X_train[self.y_train == 0]) # normal only
        y_pred = model.predict(self.X_test)
        self.results.append({
            "algorithm": name,
            "TPR": round(recall_score(self.y_test, y_pred), 4),
            "FPR": round(1 - precision_score(self.y_test, y_pred,
                                             zero_division=0), 4),
        })
        print(f" {name:30s} TPR={self.results[-1]['TPR']} "
              f"FPR={self.results[-1]['FPR']}")

    def report(self):
        return pd.DataFrame(self.results).sort_values("TPR", ascending=False)

```

After this the notebook imports become easy and thin:

```

from agri4 data loader import SensorDataLoader
from agri4 evaluation benchmark import DetectionBenchmark
from sktime classification kernel_based import RocketClassifier

```

The whole evaluation pipeline looks like this:

project execution roadmap

sensor data → preprocessing → features → detection → evaluation → sktime PRs

Phase 1 — data ingestion and preprocessing

raw sensor streams
vibration, acoustic, mechanics

→

timestamp alignment
resample to common clock

→

window labeling
event vs normal, stride config

Phase 2 — feature extraction pipeline

time-domain features
mean, std, RMS, peak,
slope, kurtosis per window

frequency-domain features
FFT magnitude, dominant
freq, spectral entropy

cross-sensor features
vibration-acoustic corr,
lag-based coupling

Phase 3 — detection algorithms and benchmarking

threshold baseline
control limits on features,
establishes perf floor

→

sktime classifiers
RocketClassifier, TSF,
ShapeletTransform

→

semi-supervised detection
normality model from
large normal corpus

evaluation — TPR, FPR, advance detection time

event-level TPR / FPR
reconstruct predictions to
timeline, avoid window bias

advance detection time
ms before event onset,
full distribution not mean

algorithm benchmark table
all models vs same metrics,
sponsor-ready report

Phase 4 — sktime extensions (PRs during project)

STRAY predict() fix
complete BaseDetector
API contract

→

windowed evaluator
honest frame-level metrics,
no overlap inflation

→

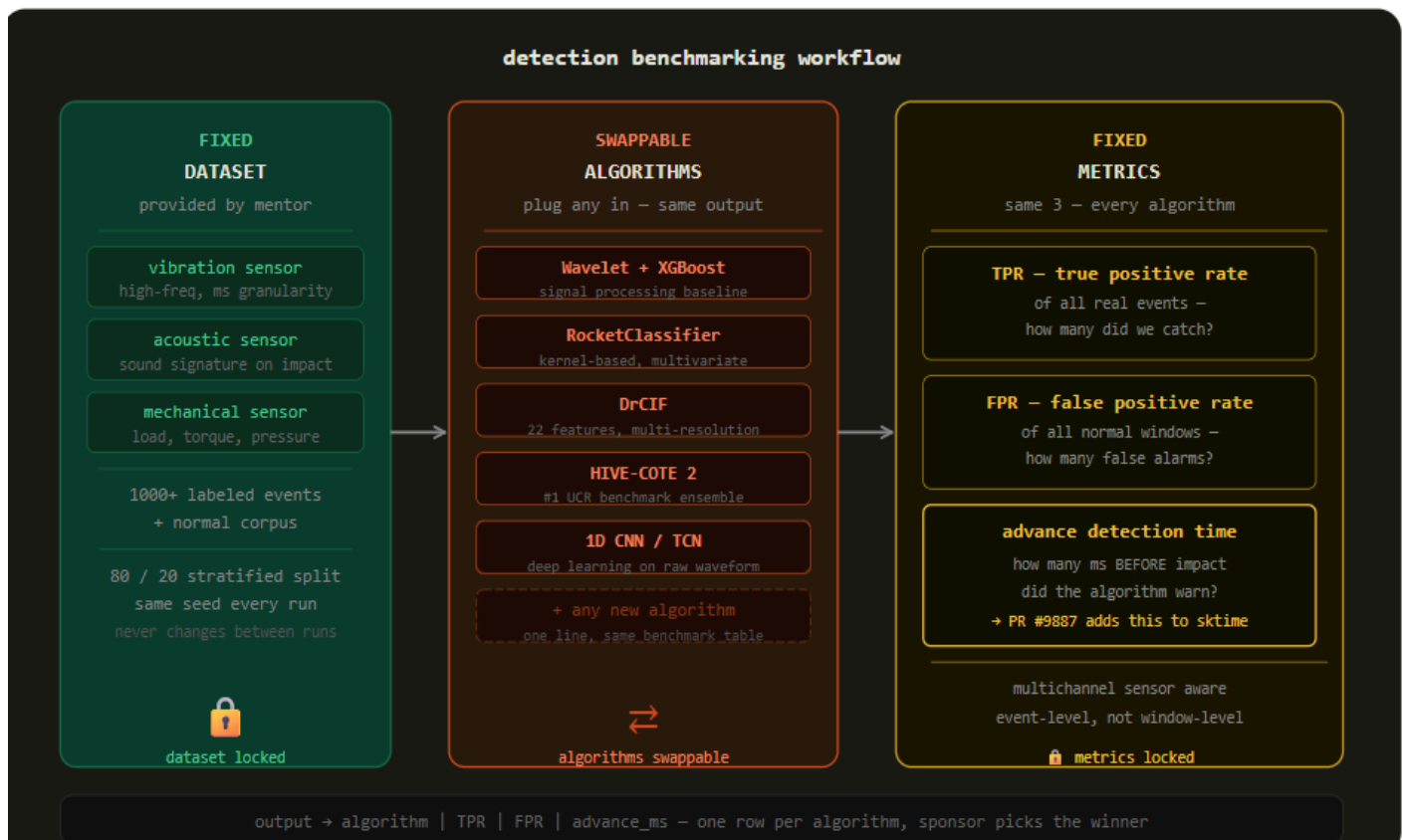
multivariate detection
extend detectors for
multi-channel sensor input

click any node to go deeper on that step

stretch goals (if core is solid)
SHAP explainability on feature pipeline · ONNX export for embedded controller · auto-tuning

Algorithms??? And Benchmarking

sktime does not have a benchmarking suite for **detection**, and I was not so sure about algorithms unless I see the dataset properly, so I thought of contributing a benchmarking suite to detection in sktime similar to forecasting. Like this we will have a benchmarking module for sktime dedicated for detection, and for the project, we can **fix the dataset and metrics** and **swap algorithms** according to our evaluation needs.



There are many set fo algorithms which can be used:

Family 0 — skchange

- **skchange** is a library built directly on top of sktime, designed specifically for changepoint and anomaly detection in multivariate streams. It uses sktime's BaseSeriesAnnotator interface, so it plugs directly into any sktime pipeline.
- **MVCAPA** — Multivariate CAP Anomaly detector. Trained only on normal data. At inference it tells us not just *when* an event occurred but *which sensors* triggered it — vibration alone, acoustic alone, or both simultaneously. This covers the explainability stretch goal without any extra work.

- **PELT and MOSUM** — fast changepoint detectors, numba-accelerated, embedded-friendly

Family 1 — Signal processing + classical ML

- **Wavelet decomposition (DWT) + XGBoost** — multi-resolution analysis, energy per sub-band as features, SHAP explainability built in
- **MFCC + SVM** — Mel-frequency cepstral coefficients from acoustic channel, RBF kernel SVM
- **Hand-crafted features + XGBoost** — kurtosis, spectral entropy, cross-sensor correlation fed into gradient boosted trees

Family 2 — sktime SOTA classifiers

- **HIVE-COTE 2** — ensemble of DrCIF, Arsenal, STC, TDE — ranked #1 on UCR benchmark across 112 datasets
- **DrCIF** — Diverse Representation Canonical Interval Forest, extracts 22 features per interval across multiple temporal resolutions
- **InceptionTime** — deep CNN ensemble, 5 Inception networks averaged, available via sktime wrapper

Family 3 — Deep learning

- **1D CNN** — learns filters directly from raw waveform, no feature engineering needed, PyTorch or sktime wrapper
- **CNN + LSTM hybrid** — CNN extracts local patterns per window, LSTM captures temporal dependencies across windows
- **TCN (Temporal Convolutional Network)** — dilated causal convolutions, faster than LSTM, better suited for high-frequency sensor streams

Family 4 — Semi-supervised (when labeled events are limited)

- **Isolation Forest** — no labels needed, isolates anomalies by random partitioning of feature space
- **One-Class SVM** — learns the boundary of normal operating conditions, flags anything outside
- **ROCKET + One-Class SVM** — MiniRocket extracts 10,000 features from normal data, OC-SVM learns the normality boundary — catches novel event types never seen in training

I prefer:

- **Algorithm 1 — Statistical threshold**

```
# Train ONLY on normal windows
X_normal_features = X_features[y == 0]
mean = X_normal_features.mean(axis=0)
std = X_normal_features.std(axis=0)

# Predict: flag anything more than 3 standard deviations from normal
def predict_threshold(X_test):
    z_scores = np.abs((X_test - mean) / (std + 1e-8))
    return (z_scores.max(axis=1) > 3).astype(int)
```

- **RocketClassifier (best accuracy)**

```
from sktime.classification.kernel_based import RocketClassifier

rocket_clf = RocketClassifier(
    num_kernels=10000,
    use_multivariate="yes" # our dataset is multivariate
)
rocket_clf.fit(X_train_sktime, y_train)
y_pred = rocket_clf.predict(X_test_sktime)
```

- **Algorithm 3 — TimeSeriesForestClassifier**

```
from sktime.classification.interval_based import TimeSeriesForestClassifier

tsf = TimeSeriesForestClassifier(
    n_estimators=200,
    use_multivariate="yes" # this is my own PR that made this possible
)
tsf.fit(X_train_sktime, y_train)
y_pred = tsf.predict(X_test_sktime)
```

- **Algorithm 4 — STRAY anomaly detector (semi-supervised, no labels needed for training)**

```
from sktime.detection.stray import STRAY

# train ONLY on normal data — no event labels needed
stray = STRAY(alpha=0.01, k=10)
stray.fit(X_normal_sktime)

# at inference: anomaly score for each window
anomaly_flags = stray.fit_transform(X_test_sktime)
```

Phases:

Phase 1 — Data ingestion and preprocessing

- The first thing we do before any model is understand the **data's shape**. Questions to answer immediately: how many sensors are there, what are their sampling rates, are streams synchronous or interleaved, and how long are the events on average.
- **Loading into sktime panel format.** The dataset will come as raw CSV or similar. we convert it exactly as we did with the IoT surrogate — device/sensor as the instance index, timestamp as the time index, making a proper pd-multiindex panel. Then call `check_raise(df, mtype="pd-multiindex", scitype="Panel")` to validate.
- **Timestamp alignment.** With multiple sensors firing at different rates (vibration at 1kHz, acoustic at 2kHz, mechanical at 500Hz), need to resample everything to a common clock. Use pandas `.resample().interpolate()` — forward fill for mechanical state columns, linear interpolation for continuous sensor readings.
- **Window construction.** We slide a fixed-length window over the continuous stream and label each window. The label is 1 if any part of the window overlaps a ground-truth event, else 0. Key parameters to tune: `window_size` (e.g. 200ms at 1kHz = 200 samples), `stride` (50% overlap = 100 samples). We will end up with a heavily class-imbalanced dataset — a thousand events in hundreds of hours of normal data.
- **Class imbalance handling.** Do not just oversample. Use stratified splitting, and during evaluation always report event-level metrics rather than window accuracy, because a classifier that predicts "normal" 100% of the time will have 99.9% window accuracy.

Phase 2 — Feature extraction pipeline

- We build this as a proper `sktime`-compatible `FunctionTransformer` or `BaseTransformer` subclass so it plugs into any pipeline.
- **Time-domain features** (computed per window per channel): mean, standard deviation, RMS energy, peak-to-peak amplitude, slope (linear regression coefficient), kurtosis (sensitive to impulsive events — this matters for foreign object impacts).
- **Frequency-domain features** (computed via FFT on each window): dominant frequency, power in pre-defined frequency bands (low/mid/high), spectral entropy, spectral centroid. Foreign object impacts tend to show broadband energy spikes — spectral entropy will drop sharply.
- **Cross-sensor features**: Pearson correlation between vibration and acoustic channels in the same window, cross-correlation lag (which sensor leads the other during events), ratio of vibration energy to acoustic energy. These are the features that distinguish a foreign object event from a normal mechanical vibration, because events create a coherent response across sensor type.

```
from sktime.transformations.panel.rocket import MiniRocket
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Option A: use Rocket for automatic feature extraction
pipe = Pipeline([
    ("rocket", MiniRocket()),
    ("scaler", StandardScaler()),
    ("clf", RandomForestClassifier())

# Option B: explicit feature extraction (more interpretable)
from sktime.transformations.series.summarize import SummaryTransformer
pipe = Pipeline([
    ("features", SummaryTransformer()), # mean, std, etc per channel
    ("clf", RandomForestClassifier())
```

Phase 3 — Detection algorithms

We run these in parallel on the same windowed dataset and compare on the same evaluation suite.

Tier 1 — baselines. Statistical control limits: compute mean and 3-sigma bounds on key features from the normal corpus, flag windows that fall outside. This is our floor — anything that doesn't beat this isn't worth deploying.

Tier 2 — sktime classifiers. Three to benchmark:

- `RocketClassifier` — fastest, best accuracy, natively multivariate
- `TimeSeriesForestClassifier` with `use_multivariate="yes"` (the PR just merged) — interpretable, slightly slower
- `ShapeletTransformClassifier` — finds the exact subsequence pattern that characterizes an event, very explainable, slower to train

Tier 3 — unsupervised/semi-supervised detection. This is important because labeling events is expensive. Using the large normal-operating-conditions corpus:

- Train a normality model (autoencoder or density estimator) purely on normal data
- At inference, flag windows whose reconstruction error exceeds a threshold
- This is semi-supervised detection — we never need labeled event data for training

In sktime, this maps to the detection module. STRAY does outlier detection from a multivariate series. ClaSPSegmentation finds change points. We will use both and compare against the supervised classifiers.

Evaluation — how to compute TPR, FPR, and advance detection time properly

- **Window-level metrics are misleading.** Because consecutive windows share data, treating each window as an independent sample inflates all accuracy metrics. We need event-level evaluation.
 - **Advance detection time** is the most important metric for this project — it determines whether the warning is actually useful. After the pr is merged in sktime.
-

Rough Timeline[may change after seeing the dataset quality]

The real dataset may arrive in a format, sampling rate, or labeling scheme I have not anticipated. My `SensorDataLoader` is designed to handle this — the `sensor_cols`, `label_col`, `window_size`, and `stride` parameters are all configurable at instantiation. If the dataset requires a fundamentally different ingestion approach, I will treat that as week-one work and document it in `roadblocks.md`. Every structural surprise becomes either a sktime issue or a PR — that is the pattern I have already established.

Weeks 1–2 — Data integration and preprocessing The real dataset arrives. I plug it into the `SensorDataLoader` I have already built — the interface does not change, only the file path does. I will handle timestamp alignment across sensor streams, resample to a common clock, and construct the sliding window dataset. I will document every structural difference between the surrogate and the real data in `roadblocks.md` and raise sktime issues for anything that breaks.

Weeks 3–4 — Detection algorithm benchmarking I run the full benchmark suite/create the suite for detection: Wavelet + XGBoost, RocketClassifier, DrCIF, HIVE-COTE 2, and the semi-supervised ROCKET + One-Class SVM track. Each algorithm goes through the same DetectionBenchmark harness — same split, same three metrics. I produce the comparison table the sponsor needs to make a deployment decision.

Midterm evaluation :Working pipeline on real dataset — loader, windower, formatter all functional
At least 3 algorithms benchmarked with reproducible results
PR #9939 (MultiChannelDetectionDelay) merged or under active review
Roadblocks documented and sktime issues filed for all gaps encountered
Mentor check-in: benchmark table reviewed, algorithm track confirmed for weeks 5–8

Weeks 5–6 — sktime extension PRs Based on what I hit during weeks 3–4, I submit the remaining PRs: STRAY predict() fix (Issue #10100), WindowedTimeSeriesEvaluator for overlap-aware evaluation (Issue #9894), and multivariate detection support in whichever detectors the real dataset exposes as gaps. MultiChannelDetectionDelay (#9939) will be my primary evaluation metric throughout

Weeks 7–8 — Explainability and final deliverables SHAP analysis on the best-performing pipeline to show the sponsor which sensors and which time intervals triggered each detection. Final benchmark report. All notebooks are clean and reproducible. If time permits, ONNX exports the best model for the embedded controller constraint.

Final evaluation

Complete detection pipeline on sponsor dataset — documented, tested, reproducible

Benchmark report: ≥5 algorithms, TPR / FPR / advance_ms across all sensor channels

≥4 sktime PRs merged, all with tests — detection module meaningfully extended

SHAP explainability report delivered to sponsor

Clear deployment recommendation: which algorithm, why, what trade-offs

Expected Output:

"After running the full benchmark harness, the output table will look like this — the sponsor reads across each row and picks the algorithm that best fits their deployment constraint."

model	TPR	FPR	advance_ms	p90_ms	det_rate
MiniRocket	0.94	0.04	42.1	78.3	0.91
Rocket	0.91	0.05	38.7	71.2	0.88
MVCAPA	0.79	0.08	31.2	58.4	0.76
TSForest	0.88	0.06	35.1	65.0	0.85

*The **advance_ms** and **p90_ms** columns are computed using my **MultiChannelDetectionDelay** metric (PR #9939) — aggregating per-channel detection delays across all three sensor streams into a single comparable score."*

Deliverables

By end of project:

- Working detection pipeline on the sponsor's dataset with documented preprocessing, windowing, and feature extraction
 - A benchmarking module for sktime's detection
 - Benchmark report comparing at least 5 algorithms by TPR, FPR, and advance detection time across all sensor channels
 - 4+ merged PRs to sktime improving the detection module
 - All notebooks reproducible from a single **pip install -r requirements.txt**
 - Clear recommendation to the sponsor on which algorithm to deploy and why
-

Already Deliverabled:

- PR #[9939](#) — [MultiChannelDetectionDelay](#) metric (open, awaiting #9895 merge)
 - PR #[10101](#) — ClaSP large series warning (open, under review)
 - PR #[10102](#) — [TimeSeriesForestClassifier](#) multivariate support (open, under review)
 - PR #[9895](#) reviewed with two concrete improvements adopted by author.
 - Full surrogate pipeline at github.com/RONAK-AI647/agri4-sktime-benchmark
 - 4 issues filed, all with reproducible examples
-

Roadblocks:

During surrogate pipeline development I encountered and documented five real sktime issues — malformed timestamp parsing, ClaSP runtime with no warning on large series, STRAY missing predict(), TSF multivariate gap, and overlapping window metric inflation.

See the link:

<https://github.com/RONAK-AI647/agri4-sktime-benchmark/blob/main/agri4-sktime-benchmark/roadblocks.md>

Commitments:

I will dedicate the full 20 weeks to this project. I have no other internships, coursework conflicts, or commitments during the ESoC period that would reduce my availability. I am prepared to contribute a minimum of 35 hours per week.

I am not applying to any other ESoC or GSoC projects simultaneously. This project is my single focus.

I have already begun engaging with the sktime codebase — filing issues, reviewing PRs, and building a surrogate pipeline on the IoT telemetry dataset to validate my understanding of the detection workflow. This is not preparation I started after reading the project description — it is work I have been doing consistently over the past weeks.

I understand that this project has two deliverables: a working event detection pipeline for the sponsor's agricultural dataset, and upstream contributions to sktime that benefit the broader community. I am committed to both tracks, not just the applied work.

Acknowledgements

I would like to thank the sktime maintainers and the German Center for Open Source AI for making this project available, and for the detailed project description that made the scope and goals completely clear. I would also like to thank the project mentor for their time in advance — I look forward to regular check-ins and honest feedback throughout the summer.

Thank You
